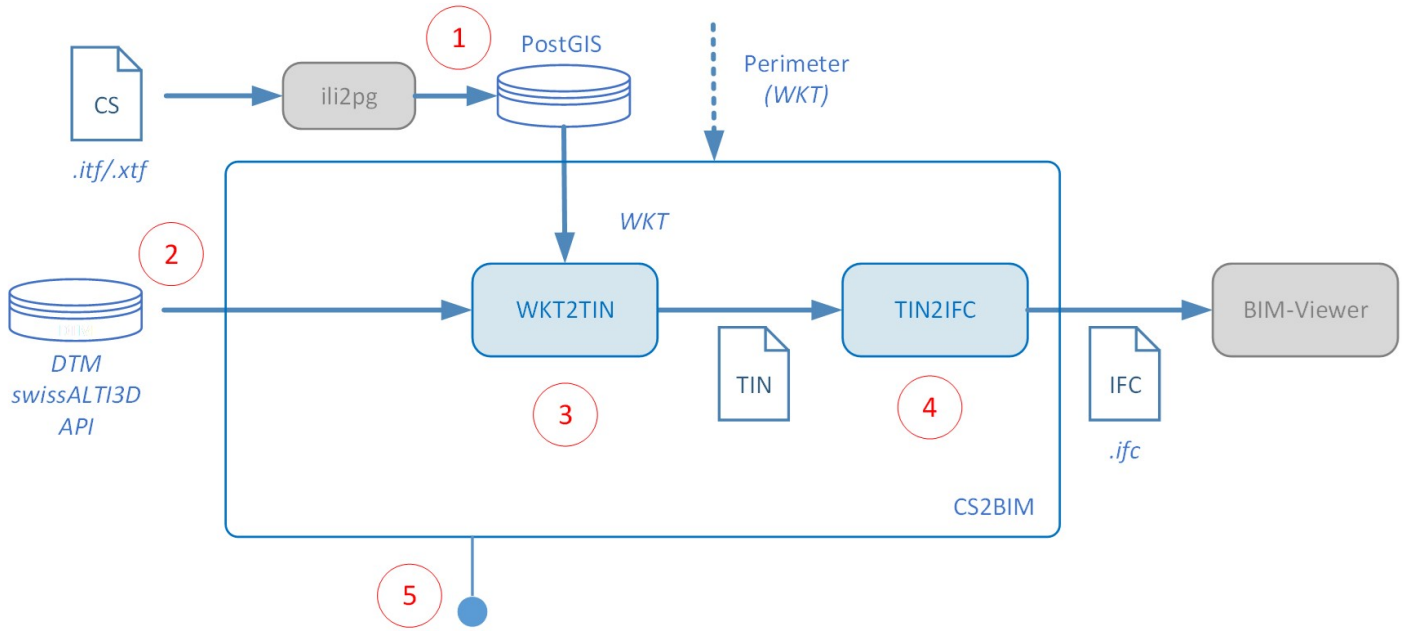
[[*TOC*]]

cs2bim project description

The Conference of Cantonal Geoinformation and Cadastral Offices (KGK) has launched a research project Cadastral Surveying Data to Building Information Modelling (CS2BIM). The Institute of Virtual Design and Construction and the Institute of Geomatics at the University of Applied Sciences Northwestern Switzerland (FHNW) have developed a service based on open source libraries. The service contains the following major components:

- Import and processing of Interlis data by the ili2pg components and storage in a PostGIS database. The vector data is read from WKT format.
- Processing of the terrain model data and creation of the resulting 3D surfaces
- Export of the objects to IFC format using the IfcOpenShell component

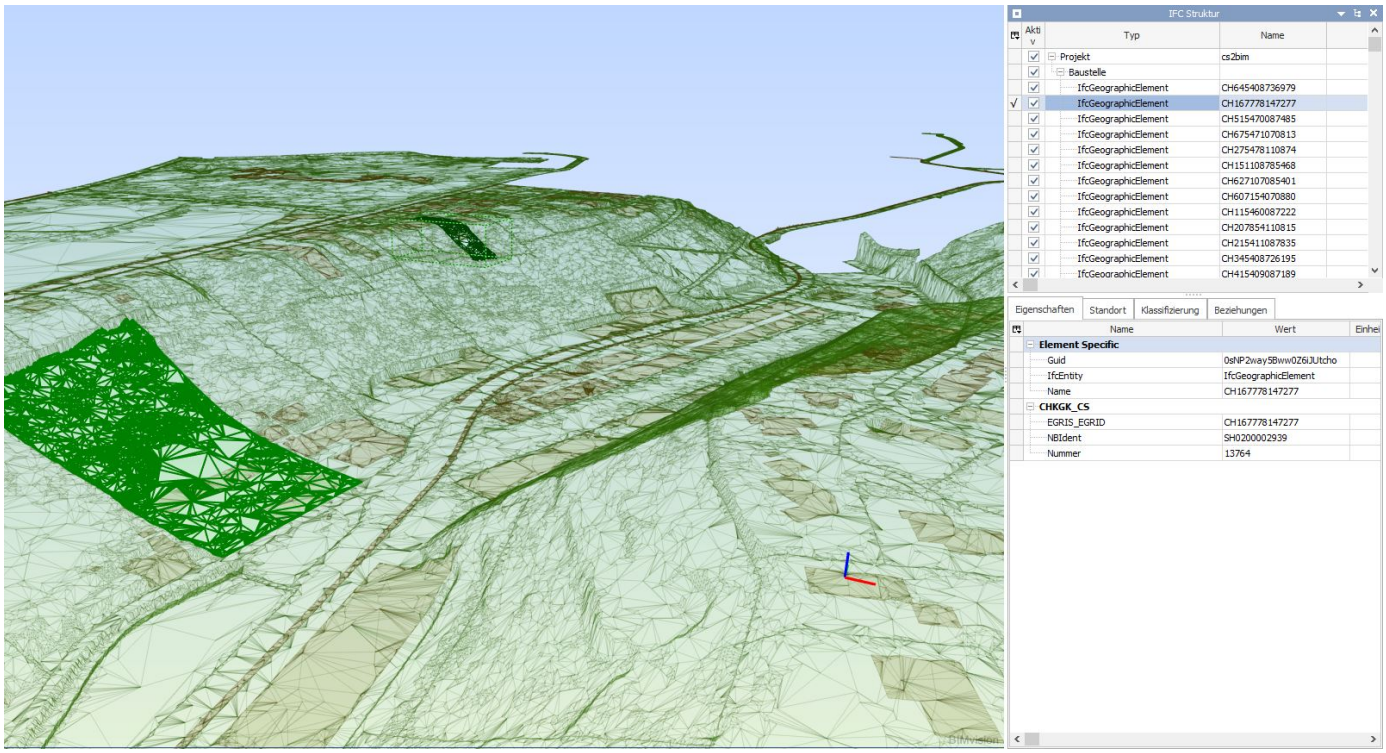


{width=600}

The diagram shows the system architecture and all major components of the implementation. Description of the compontens:

component	name	description
1	database	The cadastral data is imported in INTERLIS (.xtf or itf) format via the standard component ili2pg (swisstopo) and made available in a PostGIS database for further processing. The component database is not covered within the code
2	swisstopo database	The digital terrain model (DTM) is loaded with the API service to swissalti3d data.
3	wkt2tin	Component for creating 3D surfaces by projecting 2D vector objects (usually polygons) onto a terrain model. As a result of this component, mesh surface geometries are available for each object instance (of the cadastral information).
4	tin2ifc	This component writes the resulting IFC file. The open library IfcOpenShell is used for this purpose.
5	API & configuration	In this service package, control the individual components into a 'CS2BIM' service. The service is build in an docker container.

Resulting IFC Files



{width=1000}

Example files can be downloaded with the following links.

ID	name	link

ID	name	link
1	200mx200m.ifc	https://drive.switch.ch/index.php/s/YOgygwb3ZqmG44v
2	200mx200m_con.ifc	https://drive.switch.ch/index.php/s/2pivcwXtneYqSdY
3	200mx200m_int.ifc	https://drive.switch.ch/index.php/s/Us2SHISz1XuMsGf
4	500mx500m.ifc	https://drive.switch.ch/index.php/s/Us2SHISz1XuMsGf

Getting started

Modify the configuration according to your needs/environment. Details about configuration see below.
Prerequisites:

- The PostGIS database with the cadastral data is available (connection with psychopg is possible)
- The service to get terrain data is available

Build docker image

```
docker build -t cs2bim-run -f Dockerfile . --rm
```

Run docker image

```
docker run -e IFC_VERSION=[cs2bim.enum.ifc_version.IfcVersion] -e NAME=[str] -e POLYGON=[wkt] -v .:/workspace/output --name cs2bim-run --rm cs2bim-run
```

The run parameters are:

- IFC_VERSION: Ifc version of the resulting ifc file (supported versions/values see src\cs2bim\config\ifc_version.py).
- NAME: Name of the resulting ifc file.
- POLYGON : The area in which the data is treated. The polygon must be a valid wkt string in LV95.

Example:

- docker run -e IFC_VERSION="IFC4" -e NAME="Test" -e POLYGON="POLYGON((2689114 1285136,2689143 1285192,2689170 1285159,2689114 1285136))" -v .:/workspace/output --name cs2bim-run --rm cs2bim-run

After you run the docker container successfully there will be a new output ifc file inside the folder from where you started the container.

Important: If you change the config.yml, the container must be rebuilt to make it work.

Getting started dev

Build and run docker container or build and open container with your IDE (e.g. VSCode)

```
docker-compose -f docker-compose-dev.yml up
```

Install pip packages (Run in container at /workspace)

```
pip install --no-cache-dir --upgrade -r /workspace/requirements.txt
```

Configuration

Some properties of this python project can be configured using the config.yml file.
The configuration has different sections/topics:

- postgis configuration: Connection to the database with the cadastral data.
- swiss topo configuration: Connection to the service that provides terrain data.
- tin configuration: Configurations for the creation and treatment of tins.
- ifc configuration: Configurations of the resulting ifc file.

Configuration parameters (overview)

Line Number	Key	Type	Values	Example
0	logging_level	str	NOTSET; DEBUG; INFO; WARN; ERROR; CRITICAL	"cs2bim"
---	---	---	---	---
1	db.dbname	str	?	"cs2bim"
2	db.user	str	?	"postgres"
3	db.host	str	?	"host.docker.internal"
4	db.port	int	?	5432
5	db.password	str	?	"xxx"
6	db.schema	str	?	"cs2bim"
---	---	---	---	---
7	dtm.stac_api	str	?	"https://data.geo.admin.ch/api/stac/v0.9/collections/ch.swisstopo.swissalti3d/items"
---	---	---	---	---
8	tin.grid_size	float	0.5;2	0.5
9	tin.max_height_error	float	TODO	0.05
---	---	---	---	---
10	ifc.author	str	?	"author"
11	ifc.version	str	?	"1.0"
12	ifc.application_name	str	?	"cs2bim"
13	ifc.project_name	str	?	"Project A"
14	ifc.geo_referencing	GeoReferencing	LO_GEO_REF_30; LO_GEO_REF_40; LO_GEO_REF_50	LO_GEO_REF_30
15	ifc.triangulation_representation_type	TriangulationRepresentationType	TESSELLATION; BREP	BREP
16	ifc.feature_classes	map	---	---
17	ifc.feature_classes.*FeatureClassKeyX*.sql	str	*Path to sql file*	"sql/parcels.sql"
18	ifc.feature_classes.*FeatureClassKeyX*.entity_type	IfcElementEntityType	IFC_GEOGRAPHIC_ELEMENT	IFC_GEOGRAPHIC_ELEMENT
19	ifc.feature_classes.*FeatureClassKeyX*.attributes	list	---	---
20	ifc.feature_classes.*FeatureClassKeyX*.attributes.*ListElementX*.attribute	str	?	"Name"
21	ifc.feature_classes.*FeatureClassKeyX*.attributes.*ListElementX*.column	str	?	"egris_egrid"
22	ifc.feature_classes.*FeatureClassKeyX*.properties	list	---	---

Line Number	Key	Type	Values	Example
23	ifc.feature_classes.*FeatureClassKeyX*.properties.*ListElementX*.name	str	?	"Property"
24	ifc.feature_classes.*FeatureClassKeyX*.properties.*ListElementX*.set	str	?	"PropertySet"
25	ifc.feature_classes.*FeatureClassKeyX*.properties.*ListElementX*.column	str	?	"property_column"
26	ifc.feature_classes.*FeatureClassKeyX*.spatial_structure.entity_type	IfcSpatialStructureEntityType	IFC_SITE	IFC_SITE
27	ifc.feature_classes.*FeatureClassKeyX*.spatial_structure.attributes	list	---	---
28	ifc.feature_classes.*FeatureClassKeyX*.spatial_structure.attributes.*ListElementX*.attribute	str	?	"Name"
29	ifc.feature_classes.*FeatureClassKeyX*.spatial_structure.attributes.*ListElementX*.value	str	?	"Site"
30	ifc.feature_classes.*FeatureClassKeyX*.group_columns	list[str]	?	"group_column"
31	ifc.feature_classes.*FeatureClassKeyX*.color_definition.r	float	0.0 - 1-0	0.1
32	ifc.feature_classes.*FeatureClassKeyX*.color_definition.g	float	0.0 - 1-0	0.5
33	ifc.feature_classes.*FeatureClassKeyX*.color_definition.b	float	0.0 - 1-0	0.5
34	ifc.feature_classes.*FeatureClassKeyX*.color_definition.a	float	0.0 - 1-0	0.3
35	ifc.feature_classes.*FeatureClassKeyX*.groups	map	---	---
36	ifc.feature_classes.*FeatureClassKeyX*.groups.*IfcGroupKeyX*.entity_type	IfcGroupEntityType	IFC_DISTRIBUTION_SYSTEM, IFC_DISTRIBUTION_CIRCUIT, IFC_BUILDING_SYSTEM, IFC_STRUCTURAL_ANALYSIS_MODEL, IFC_ZONE	IFC_DISTRIBUTION_SYSTEM
37	ifc.feature_classes.*FeatureClassKeyX*.groups.*IfcGroupKeyX*.attributes	list	---	---
38	ifc.feature_classes.*FeatureClassKeyX*.groups.*IfcGroupKeyX*.attributes.*ListElementX*.attribute	str	?	"Name"
39	ifc.feature_classes.*FeatureClassKeyX*.groups.*IfcGroupKeyX*.attributes.*ListElementX*.value	str	?	"Group"

Types

Some parameters can only be configured with predefined values (types), because these values are referenced in the code. To guarantee a proper configuration and execution of the code, these predefined values (types) are defined as constants in different modules/classes in the python code.

The following types are defined:

- GeoReferencing -> cs2bim.config.geo_referencing.py\
- TriangulationRepresentationType -> cs2bim.geometry.triangulation.py\
- IfcElementEntityType -> cs2bim.ifc.entity.ifc_element.py\
- IfcSpatialStructureEntityType -> cs2bim.ifc.entity.ifc_spatial_structure.py\
- IfcGroupEntityType -> cs2bim.ifc.entity.ifc_group.py

IFC configuration

In this section of the configuration you can make some general definitions about the resulting ifc file and you can define the feature classes that are generated and exported as ifc entities.

Below some of the parameters are explained.

Geo Referencing

You can provide the so called "Level of Georeferencing" (LoGeoRef), according to *"Clemen, C., Görne, H., 2019. Level of Georeferencing (LoGeoRef) using IFC for BIM. Journal of Geodesy, Cartography and Cadastre, 10/2019, S. 15-20. ISSN: 1454-1408"*.
The different levels represent different methods of defining informations about georeferencing in IFC.
Supported values are LO_GEO_REF_30, LO_GEO_REF_40, LO_GEO_REF_50.

Triangulation Representation Type

You can define the IFC geometry type that is used to represent the TIN geometry.
Supported values are TESSELLATION, BREP (TESSELATION is recommended).

Feature Classes

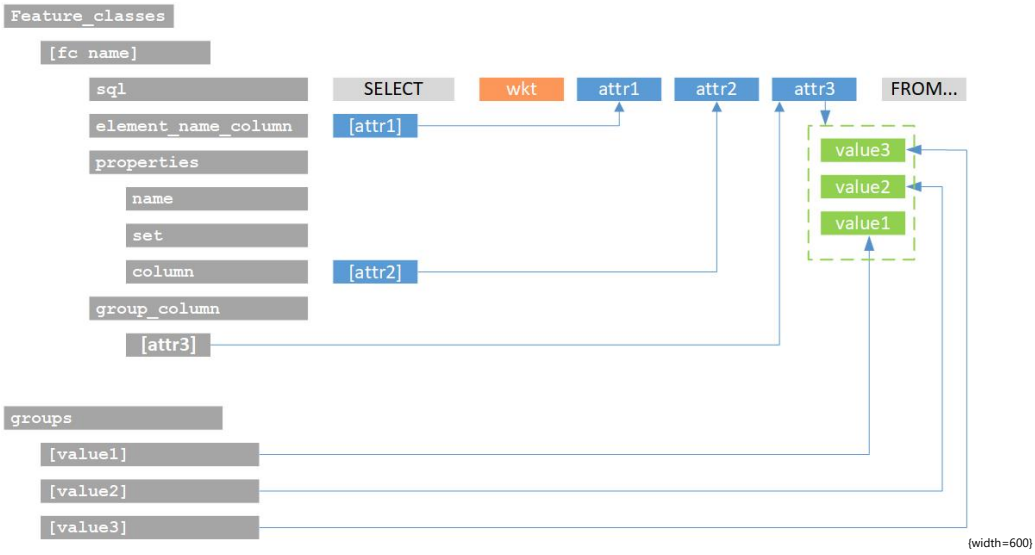
A "Feature Class" is the definition of a set of objects that are exported in an IFC entity with common definitions.
The main configurations of a feature class include:

- sql: A SQL query that selects objects in the GIS database, returning a geometry (must be an area) and some other attributes for each object.
- entity_type: The IFC entity, to which all selected objects of the feature class are exported to.
- properties: Any number of property definitions that are exported as IFC properties/property sets.
- group_columns: Any number of IFC group assignments.
- spatial_structure: The IFC spatial structure, that appends all objects of the feature class.
- colour_definition: An IFC colour definition

SQL

For each feature class you have to provide a sql for querying the data(17). With the query you are selecting the cadastral data (with area geometry type). The sql query requires to take a polygon wkt as parameter "%(polygon)s" and return a column named "wkt" with wkt string values. To guarantee a correct processing it is important to check that the sql also delivers all columns that are additionally configured for the according feature class. This can be multiple columns for attributes(21), properties(25) or groups(30).

The following schema shows the relationship between the attributes defined by the sql query and their linking to the configuration.



{width=600}

Groups

Every exported object can be assigned to a group (zero to multiple). The assignment is defined by an attribute value (of the sql query). For each attribute value, that is used as a group assignment, there should be a group configuration.
For each group configuration the system is creating an ifc group according to the configured parameters (entity_type, predefined_type, object_type).
When there is no group configuration for an assigned value, the system will create a simple ifc group entity without any special attributes.

When defining a group you can use "." to create nested group structures. (IfcGroupKey)
By default all IfcGroups are generated using the IfcGroup entity type. Defining other types of groups can be done by creating a new group config referencing the IfcGroupKey in the configuration file(32).

Example

config.yml

```
...
feature_classes:
    feature_class_x:
        sql: "sql/feature_class_x.sql"
        element_name_column: "name_column"
        properties:
            - name: "Property"
            column: "property_column"
            set: "PropertySetName"
        group_columns:
            - "group"
groups:
    Amtliche Vermessung.Feature-Klassen:
        entity_type: IFC_DISTRIBUTION_SYSTEM
        object_type: ""
        predefined_type: "NOTDEFINED"
    Amtliche Vermessung.Feature-Klassen.x:
        entity_type: IFC_DISTRIBUTION_SYSTEM
        object_type: "ObjectType"
        predefined_type: "NOTDEFINED"
...
```

sql/feature_class_x.sql

```
with perimeter as (
    select
        ST_GeomFromText(%(polygon)s, 2056) as geom
)
select
    ST_AsText(ST_CurveToLine(geometrie, 1)) as wkt,
    nbident as name_column,
    nummer as property_column,
    CASE
        WHEN x THEN 'Amtliche Vermessung.Feature-Klassen.x'
        ELSE 'Amtliche Vermessung.Feature-Klassen.y'
    END as group
from
    cs2bim.liegenschaft l
    left join cs2bim.grundstueck g on (l.liegenschaft_von = g.t_id)
    join perimeter on ST_Intersects(geometrie, perimeter.geom)
```

Useful postgis functions:

ST_GeomFromText -> Constructs a PostGIS ST_Geometry object
ST_AsText -> Returns the OGC WKT representation of the geometry
ST_CurveToLine -> Converts a given geometry to a linear geometry
ST_Intersects -> Returns true if two geometries intersect. Geometries intersect if they have any point in common. ST_Contains -> Returns true if the first geometry contains the second.

Code structure

config

Contains all files needed for the configuration of the main processing step.

geometry

Holds classes that hold information about certain geometry objects.

ifc

Builds an Ifc file using the ifcopenshell library based on a model object.

service

There are two services available. A postgis service to query a postgis database and a dtm service to download terrain models.

tin

The tin package allows to create triangulations by clipping terrain models with wkt strings.

main.py

To execute the main function you need to provide three parameters: IFC_VERSION, NAME and POLYGON. The program executes the following steps:

1. Load configuration file
2. Create model --> TO CHEK: What is a model???
3. Build model
4. Save model

Known Issues

- Entity types that are only supported in one of the two allowed ifc versions (4, 4x3) are not supported (e.g. IfcBuiltSystem)
- Only one supported type of feature class data: All feature classes are processed the same way and are implemented to represent a surface that is projected to the terrain.